

문제 01 · 객관식

완전 탐색(brute force)을 가리키는 다른 이름으로 볼 수 없는 것은?

- ① 탐욕적 선택 기법(greedy method)
- ② 전수 조사(exhaustive search)
- ③ 무차별 대입 기법
- ④ 순진한 방법(naive method)
- ⑤ 모든 경우 시도 기법

정답 및 해설 01

완전 탐색(brute force)을 가리키는 다른 이름으로 볼 수 없는 것은?

정답 ① 탐욕적 선택 기법(greedy method)

해설

완전 탐색은 가능한 경우를 빠짐없이 확인하는 전략이며 exhaustive search, naive method, 무차별 대입 등으로 불린다. 탐욕적 선택은 매 순간 가장 좋아 보이는 것만 고르고 되돌아보지 않는 전혀 다른 전략이다.

문제 02 · 객관식

완전 탐색에 대한 설명으로 옳지 않은 것은?

- ① 모든 경우를 확인하므로 정답을 놓치지 않는다
- ② 복잡한 수학 없이 반복문만으로 구현할 수 있다
- ③ 경우의 수가 많아도 실행 시간이 거의 늘지 않는다
- ④ 입력이 커지면 확인할 후보가 급격히 늘어나는 문제도 있다
- ⑤ 고급 알고리즘을 설계할 때 출발점으로 삼을 수 있다

정답 및 해설 02

완전 탐색에 대한 설명으로 옳지 않은 것은?

정답 ③ 경우의 수가 많아도 실행 시간이 거의 늘지 않는다

해설

전체 시간은 후보 수와 후보 하나의 검사 비용에 달려 있다. 순차 탐색처럼 후보 수가 선형으로 늘어나는 문제도 있고, 순열처럼 급격히 늘어나는 문제도 있다. 후보가 많아져도 시간이 거의 늘지 않는다는 ③은 옳지 않다.

문제 03 · 객관식

0~9의 숫자를 중복해서 쓸 수 있고 앞자리 0도 허용하는 4자리 자물쇠 비밀번호는 모두 몇 가지인가?

- ① 10,000가지
- ② 40가지
- ③ 5,040가지
- ④ 1,000가지
- ⑤ 100,000가지

정답 및 해설 03

0~9의 숫자를 중복해서 쓸 수 있고 앞자리 0도 허용하는 4자리 자물쇠 비밀번호는 모두 몇 가지인가?

정답 ① 10,000가지

해설

각 자리마다 10가지이므로 $10 \times 10 \times 10 \times 10 = 10^4 = 10,000$ 가지다. 자릿수가 하나 늘 때마다 경우의 수는 10배가 된다.

문제 04 · 객관식

다음 코드에서 출력되는 count 값은?

code.py

```
1 secret = "8234"
2 count = 0
3 for d1 in range(10):
4     for d2 in range(10):
5         for d3 in range(10):
6             for d4 in range(10):
7                 attempt = f"{d1}{d2}{d3}{d4}"
8                 count += 1
9                 if attempt == secret:
10                     print(count); exit()
```

① 8235

② 8234

③ 8236

④ 10000

⑤ 1

정답 및 해설 04

다음 코드에서 출력되는 count 값은?

code.py

```
1 secret = "8234"
2 count = 0
3 for d1 in range(10):
4     for d2 in range(10):
5         for d3 in range(10):
6             for d4 in range(10):
7                 attempt = f"{d1}{d2}{d3}{d4}"
8                 count += 1
9                 if attempt == secret:
10                     print(count); exit()
```

정답 ① 8235

해설

0000부터 순서대로 시도하며 시도할 때마다 count를 1 증가시킨다. 0000이 1회째이므로 8234는 $8234 + 1 = 8235$ 회째에 발견된다. 숫자 자체와 시도 횟수가 1만큼 차이 난다는 점이 핵심이다.

문제 05 · 객관식

완전 탐색이 정답을 놓치지 않는 근거로 가장 적절한 것은?

- ① 반복문이 언제나 정해진 횟수만큼만 돌고 끝나기 때문이다
- ② 경우의 수를 미리 계산해 두고 탐색을 시작하기 때문이다
- ③ 정답에 가까울 것 같은 후보부터 우선 확인하기 때문이다
- ④ 가능한 경우를 하나도 빠뜨리지 않고 모두 확인하기 때문이다
- ⑤ 조건을 만족하지 않는 후보를 미리 걸러내고 시작하기 때문이다

정답 및 해설 05

완전 탐색이 정답을 놓치지 않는 근거로 가장 적절한 것은?

정답 ④ 가능한 경우를 하나도 빠뜨리지 않고 모두 확인하기 때문이다

해설

가능한 후보를 빠짐없이 만들고 각 후보가 조건을 만족하는지 검사하면 정답인 후보를 놓치지 않는다. 따라서 가능한 경우를 모두 확인한다는 ④가 근거이다.

문제 06 · 객관식

이 컴퓨터는 제한 시간 안에 후보를 최대 1,000만 개 검사할 수 있다. 모든 후보의 검사 비용이 같을 때, 완전 탐색으로 시간 안에 끝남을 판단할 수 있는 정보는?

- ① 입력의 크기가 미리 정해져 있지 않아 알 수 없는 경우
- ② 검사할 후보가 정확히 100만 개이다
- ③ 구해야 할 정답이 여러 개일 가능성이 있는 경우
- ④ 데이터가 미리 오름차순으로 정렬되어 있는 경우
- ⑤ 반복문을 두 겹 이상 사용할 수 없다는 제약이 있는 경우

정답 및 해설 06

이 컴퓨터는 제한 시간 안에 후보를 최대 1,000만 개 검사할 수 있다. 모든 후보의 검사 비용이 같을 때, 완전 탐색으로 시간 안에 끝남을 판단할 수 있는 정보는?

정답 ② 검사할 후보가 정확히 100만 개이다

해설

100만 개는 주어진 한도인 1,000만 개 이하이다. 다른 보기만으로는 검사할 후보 수가 한도 안에 드는지 알 수 없다.

문제 07 · 객관식

각 선택 변수를 한 겹의 반복문으로 순회하는 직접 구현에서, 중첩 반복문의 개수를 결정하는 것은?

- ① 문제에서 요구하는 정답의 개수
- ② 입력 데이터의 총 개수
- ③ 동시에 정해야 하는 변수의 개수
- ④ 사용할 수 있는 메모리의 크기
- ⑤ 리스트에 저장된 값의 최대 크기

정답 및 해설 07

각 선택 변수를 한 겹의 반복문으로 순회하는 직접 구현에서, 중첩 반복문의 개수를 결정하는 것은?

정답 ③ 동시에 정해야 하는 변수의 개수

해설

선택 변수마다 반복문을 한 겹씩 쓰는 직접 구현에서는 변수 수와 중첩 깊이가 같다. 각 후보 범위와 내부 처리 비용도 함께 확인해야 시간 복잡도를 정할 수 있다.

문제 08 · 객관식

다음 코드에서 find_divisors(10)의 반환값은?

code.py

```
1 def find_divisors(n):  
2     divisors = []  
3     for i in range(1, n + 1):  
4         if n % i == 0:  
5             divisors.append(i)  
6     return divisors
```

① [1, 2, 5, 10]

② [1, 2, 4, 5, 10]

③ [1, 2, 5]

④ [2, 5, 10]

⑤ [1, 5, 10]

정답 및 해설 08

다음 코드에서 find_divisors(10)의 반환값은?

code.py

```
1 def find_divisors(n):
2     divisors = []
3     for i in range(1, n + 1):
4         if n % i == 0:
5             divisors.append(i)
6     return divisors
```

정답 ① [1, 2, 5, 10]

해설

1부터 10까지 모두 시도하여 10을 나누어떨어지게 하는 수를 모은다. 1, 2, 5, 10이 해당하며 1과 자기 자신도 약수이므로 포함된다.

문제 09 · 객관식

다음 코드의 시간 복잡도를 가장 정확히 나타낸 것은? n 은 양의 정수 입력값이며 나머지·비교와 `append`의 평균 비용은 $O(1)$ 로 본다.

code.py

```
1 def find_divisors(n):
2     divisors = []
3     for i in range(1, n + 1):
4         if n % i == 0:
5             divisors.append(i)
6     return divisors
```

① $O(\sqrt{n})$

② $O(1)$

③ $O(\log n)$

④ $O(n^2)$

⑤ $O(n)$

정답 및 해설 09

다음 코드의 시간 복잡도를 가장 정확히 나타낸 것은? n 은 양의 정수 입력값이며 나머지·비교와 `append`의 평균 비용은 $O(1)$ 로 본다.

code.py

```
1 def find_divisors(n):
2     divisors = []
3     for i in range(1, n + 1):
4         if n % i == 0:
5             divisors.append(i)
6     return divisors
```

정답 ⑤ $O(n)$

해설

반복문이 1부터 n 까지 정확히 n 번 돈다. 참고로 $\sqrt{n}$ 까지만 검사하고 짝을 함께 구하는 방식으로 개선하면 $O(\sqrt{n})$ 이 되지만, 이 코드는 그렇게 되어 있지 않다.

문제 10 · 객관식

다음 함수에서 find_pairs([1, 3, 5, 7, 9], 10)의 반환값은?

code.py

```
1 def find_pairs(arr, K):
2     pairs = []
3     for i in range(len(arr) - 1):
4         for j in range(i + 1, len(arr)):
5             if arr[i] + arr[j] == K:
6                 pairs.append((arr[i], arr[j]))
7     return pairs
```

- ① [(3, 7)]
- ② [(1, 9), (3, 7), (5, 5)]
- ③ [(1, 9), (9, 1), (3, 7), (7, 3)]
- ④ [(1, 9), (3, 7)]
- ⑤ [(5, 5)]

정답 및 해설 10

다음 함수에서 find_pairs([1, 3, 5, 7, 9], 10)의 반환값은?

code.py

```
1 def find_pairs(arr, K):
2     pairs = []
3     for i in range(len(arr) - 1):
4         for j in range(i + 1, len(arr)):
5             if arr[i] + arr[j] == K:
6                 pairs.append((arr[i], arr[j]))
7     return pairs
```

정답 ④ [(1, 9), (3, 7)]

해설

j를 i+1부터 돌리므로 같은 쌍을 두 번 세지 않고 자기 자신과도 짝짓지 않는다. 합이 10인 쌍은 (1,9)와 (3,7) 두 개다.

문제 11 · 객관식

다음 코드에서 안쪽 반복문의 시작을 $j = i + 1$ 로 둔 이유로 가장 적절한 것은?

code.py

```
1 def find_pairs(arr, K):
2     pairs = []
3     for i in range(len(arr) - 1):
4         for j in range(i + 1, len(arr)):
5             if arr[i] + arr[j] == K:
6                 pairs.append((arr[i], arr[j]))
7     return pairs
```

- ① 인덱스가 리스트 범위를 벗어나지 않게 하기 위해
- ② 리스트를 정렬된 상태로 유지하기 위해
- ③ 같은 쌍이 두 번 세어지는 것을 막기 위해
- ④ 합이 K보다 커지는 경우를 미리 제외하기 위해
- ⑤ 시간 복잡도를 $O(n)$ 으로 낮추기 위해

정답 및 해설 11

다음 코드에서 안쪽 반복문의 시작을 $j = i + 1$ 로 둔 이유로 가장 적절한 것은?

code.py

```
1 def find_pairs(arr, K):
2     pairs = []
3     for i in range(len(arr) - 1):
4         for j in range(i + 1, len(arr)):
5             if arr[i] + arr[j] == K:
6                 pairs.append((arr[i], arr[j]))
7     return pairs
```

정답 ③ 같은 쌍이 두 번 세어지는 것을 막기 위해

해설

j 를 0부터 돌리면 (1,9)와 (9,1)이 모두 세어지고 (1,1)처럼 자기 자신과 짝을 이루는 경우도 생긴다. $i+1$ 부터 시작하면 $i < j$ 가 보장되어 각 쌍이 한 번씩만 나온다. 복잡도는 여전히 $O(n^2)$ 이다.

문제 12 · 객관식

길이 n 인 리스트의 서로 다른 두 인덱스 $i < j$ 를 골라 각 쌍을 정확히 한 번 검사한다.
 $n \geq 2$ 일 때 총 검사 횟수는?

- ① $n(n+1)/2$
- ② n^2
- ③ $n(n-1)/2$
- ④ $n-1$
- ⑤ $2n$

정답 및 해설 12

길이 n 인 리스트의 서로 다른 두 인덱스 $i < j$ 를 골라 각 쌍을 정확히 한 번 검사한다.
 $n \geq 2$ 일 때 총 검사 횟수는?

정답 ③ $n(n-1)/2$

해설

$i=0$ 이면 j 를 $n-1$ 번, $i=1$ 이면 $n-2$ 번 검사한다. 따라서 총횟수는 $(n-1)+(n-2)+\dots+1=n(n-1)/2$ 이다. j 값이 줄어드는 것이 아니라, 다음 i 에서 검사할 j 의 개수가 줄어든다.

문제 13 · 객관식

다음 코드에서 안쪽 조건 검사가 실행되는 총 횟수는?

code.py

```
1 def dice_sum(target):
2     results = []
3     for d1 in range(1, 7):
4         for d2 in range(1, 7):
5             for d3 in range(1, 7):
6                 if d1 + d2 + d3 == target:
7                     results.append((d1, d2, d3))
8     return results
```

① 216회

② 18회

③ 36회

④ 27회

⑤ 666회

정답 및 해설 13

다음 코드에서 안쪽 조건 검사가 실행되는 총 횟수는?

code.py

```
1 def dice_sum(target):
2     results = []
3     for d1 in range(1, 7):
4         for d2 in range(1, 7):
5             for d3 in range(1, 7):
6                 if d1 + d2 + d3 == target:
7                     results.append((d1, d2, d3))
8     return results
```

정답 ① 216회

해설

각 주사위는 1~6을 모두 시도하므로 안쪽 if 조건은 $6 \times 6 \times 6 = 216$ 번 검사한다. target의 값이나 조건을 만족한 결과의 개수가 달라져도 이 검사 횟수는 같다.

문제 14 · 객관식

서로 구별되는 주사위 3개의 눈이 각각 $1 \sim n$ 이다($n \geq 1$). 가능한 세 눈의 조합을 빠짐없이 한 번씩 검사하며, 한 조합 검사는 $O(1)$ 이다. 전체 시간 복잡도로 가장 알맞은 것은?

- ① $O(n \log n)$
- ② $O(n^2)$
- ③ $O(3n)$
- ④ $O(n^3)$
- ⑤ $O(3^n)$

정답 및 해설 14

서로 구별되는 주사위 3개의 눈이 각각 $1 \sim n$ 이다($n \geq 1$). 가능한 세 눈의 조합을 빠짐없이 한 번씩 검사하며, 한 조합 검사는 $O(1)$ 이다. 전체 시간 복잡도로 가장 알맞은 것은?

정답 ④ $O(n^3)$

해설

첫째 주사위 n 가지마다 둘째가 n 가지, 그 각각마다 셋째가 n 가지이다. 총 $n \times n \times n = n^3$ 개를 일정한 비용으로 검사하므로 $O(n^3)$ 이다.

문제 15 · 객관식

완전 탐색의 3단계 템플릿을 순서대로 바르게 나열한 것은?

- ① 후보 해 생성 → 해 선택 → 조건 검증
- ② 조건 검증 → 후보 해 생성 → 해 선택
- ③ 해 선택 → 후보 해 생성 → 조건 검증
- ④ 후보 해 생성 → 조건 검증 → 해 선택
- ⑤ 조건 검증 → 해 선택 → 후보 해 생성

정답 및 해설 15

완전 탐색의 3단계 템플릿을 순서대로 바르게 나열한 것은?

정답 ④ 후보 해 생성 → 조건 검증 → 해 선택

해설

가능한 경우를 만들고(생성), 문제 조건을 만족하는지 보고(검증), 통과한 것을 답으로 삼거나 최적 값과 비교한다(선택). 이 순서는 어떤 완전 탐색 문제에서도 동일하다.

문제 16 · 객관식

완전 탐색 템플릿에서 '조건 검증' 단계가 코드로 나타나는 형태로 가장 적절한 것은?

- ① 후보가 문제 조건을 만족하는지 판단하는 if 문
- ② 가능한 경우를 만들어 내는 for 문
- ③ 최적값을 담아 두는 변수의 초기화
- ④ 결과를 모아 두는 리스트의 append 호출
- ⑤ 함수의 마지막 return 문

정답 및 해설 16

완전 탐색 템플릿에서 '조건 검증' 단계가 코드로 나타나는 형태로 가장 적절한 것은?

정답 ① 후보가 문제 조건을 만족하는지 판단하는 if 문

해설

생성은 반복문이, 검증은 조건문이, 선택은 반환이나 최적값 갱신이 담당한다. 조건문은 만들어진 후보를 걸러 내는 거름망 역할을 한다.

문제 17 · 객관식

완전 탐색에서 '해 선택' 단계가 정답 하나만 필요한 경우와 최적해가 필요한 경우에 각각 어떻게 달라지는가?

- ① 하나만 필요하면 계속 비교·갱신하고, 최적해가 필요하면 즉시 반환한다
- ② 조건을 만족하는 답 하나를 찾으면 반환하고, 최적해는 계속 비교·갱신한다
- ③ 두 경우 모두 반드시 끝까지 탐색을 마친 뒤에 결과를 반환한다
- ④ 두 경우 모두 첫 번째 후보를 만나는 순간 그것을 반환한다
- ⑤ 하나만 필요하면 후보 생성 단계를 통째로 건너뛸 수 있다

정답 및 해설 17

완전 탐색에서 '해 선택' 단계가 정답 하나만 필요한 경우와 최적해가 필요한 경우에 각각 어떻게 달라지는가?

정답 ② 조건을 만족하는 답 하나를 찾으면 반환하고, 최적해는 계속 비교·갱신한다

해설

조건을 만족하는 답 하나면 충분한 문제는 찾는 즉시 return 하여 남은 탐색을 생략할 수 있다. 최적해를 구해야 하면 모든 후보를 끝까지 보며 최적값을 갱신해야 한다.

문제 18 · 객관식

완전 탐색의 후보 해 생성 단계가 반드시 지켜야 할 성질은?

- ① 생성한 후보를 정렬된 순서로 유지해야 한다
- ② 정답에 가까운 후보부터 생성해야 한다
- ③ 가능한 후보를 빠짐없이 생성해야 한다
- ④ 생성 개수가 입력 크기보다 작아야 한다
- ⑤ 생성 도중 조건을 만족하면 즉시 멈춰야 한다

정답 및 해설 18

완전 탐색의 후보 해 생성 단계가 반드시 지켜야 할 성질은?

정답 ③ 가능한 후보를 빠짐없이 생성해야 한다

해설

정답이 될 수 있는 후보가 빠지면 정답을 놓칠 수 있다. 중복 생성은 시간을 낭비하며 경우의 수를 셀 때는 중복 계수를 막아야 하지만, 모든 탐색의 정확성에 중복 제거가 반드시 필요한 것은 아니다.

문제 19 · 객관식

완전 탐색 구조가 고급 알고리즘의 출발점이 된다고 말하는 이유로 가장 적절한 것은?

- ① 고급 기법들이 완전 탐색의 비효율을 줄이는 방향으로 발전했기 때문이다
- ② 완전 탐색이 어떤 문제에서든 가장 빠른 방법으로 알려져 있기 때문이다
- ③ 고급 기법들이 완전 탐색보다 구현하기 훨씬 간단하기 때문이다
- ④ 고급 기법들이 완전 탐색과 똑같은 시간 복잡도를 갖기 때문이다
- ⑤ 완전 탐색이 정답 대신 근사한 답만 구해 주기 때문이다

정답 및 해설 19

완전 탐색 구조가 고급 알고리즘의 출발점이 된다고 말하는 이유로 가장 적절한 것은?

정답 ① 고급 기법들이 완전 탐색의 비효율을 줄이는 방향으로 발전했기 때문
이다

해설

백트래킹은 가지치기를, 분할 정복은 문제 분할을, 동적 계획법은 기억을 더해 완전 탐색의 낭비를 줄인 것이다. 기준점이 되는 구조가 완전 탐색이다.

문제 20 · 객관식

순열과 조합을 구분하는 기준으로 가장 적절한 것은?

- ① 같은 원소를 두 번 이상 뽑는 것이 허용되는가
- ② 뽑아야 하는 원소의 개수가 미리 정해져 있는가
- ③ 뽑은 결과에서 순서가 다르면 다른 경우로 볼 것인가
- ④ 원본 데이터가 오름차순으로 정렬되어 있는가
- ⑤ 결과를 리스트와 튜플 중 어느 쪽으로 저장하는가

정답 및 해설 20

순열과 조합을 구분하는 기준으로 가장 적절한 것은?

정답 ③ 뽑은 결과에서 순서가 다르면 다른 경우로 볼 것인가

해설

순서를 구별하면 순열, 구별하지 않으면 조합이다. 1등과 2등을 정하는 문제는 순열, 대표 두 명을 뽑는 문제는 조합이다.

문제 21 · 객관식

다음 코드의 출력값은?

code.py

```
1 from itertools import permutations
2 items = ['A', 'B', 'C']
3 result = list(permutations(items, 2))
4 print(len(result))
```

① 3

② 6

③ 2

④ 9

⑤ 8

정답 및 해설 21

다음 코드의 출력값은?

code.py

```
1 from itertools import permutations
2 items = ['A', 'B', 'C']
3 result = list(permutations(items, 2))
4 print(len(result))
```

정답 ② 6

해설

3개 중 2개를 순서 있게 나열하므로 $3P2 = 3 \times 2 = 6$ 이다. (A,B)와 (B,A)를 서로 다른 경우로 세기 때문에 조합보다 개수가 많다.

문제 22 · 객관식

다음 코드의 출력값은?

code.py

```
1 from itertools import combinations
2 items = ['A', 'B', 'C']
3 result = list(combinations(items, 2))
4 print(len(result))
```

① 3

② 6

③ 2

④ 9

⑤ 4

정답 및 해설 22

다음 코드의 출력값은?

code.py

```
1 from itertools import combinations
2 items = ['A', 'B', 'C']
3 result = list(combinations(items, 2))
4 print(len(result))
```

정답 ① 3

해설

3개 중 2개를 순서 없이 고르므로 $3C2 = 3$ 이다. (A,B)와 (B,A)를 같은 경우로 보기 때문에 순열의 절반이 된다.

문제 23 · 객관식

다음 중 조합(combination)으로 세어야 하는 상황은?

- ① 5명 중에서 회장과 부회장을 각각 한 명씩 정하는 경우
- ② 5개 메뉴 중 서로 다른 2개를 골라 세트를 만드는 경우
- ③ 서로 다른 숫자 3개를 이어 붙여 비밀번호를 만드는 경우
- ④ 네 개의 도시를 한 번씩 방문하는 순서를 정하는 경우
- ⑤ 달리기 경기에서 1등, 2등, 3등을 각각 정하는 경우

정답 및 해설 23

다음 중 조합(combination)으로 세어야 하는 상황은?

정답 ② 5개 메뉴 중 서로 다른 2개를 골라 세트를 만드는 경우

해설

세트에서는 초코·바닐라와 바닐라·초코가 같은 메뉴이므로 순서를 구별하지 않는다. 나머지 넷은 모두 순서가 결과를 바꾸므로 순열이다.

문제 24 · 객관식

서로 다른 n 개 중 중복 선택 없이 r 개를 골라 순서 있게 나열하는 경우의 수의 일반식은? $1 \leq r \leq n$ 이다.

- ① $r \times (r-1) \times \dots \times 2 \times 1$
- ② $n \times (n-1) \times \dots \times (n-r)$ 의 곱
- ③ $n! / (r! \times (n-r)!)$
- ④ $n \times n \times \dots \times n$ (n 을 r 번 곱함)
- ⑤ $n \times (n-1) \times \dots \times (n-r+1)$

정답 및 해설 24

서로 다른 n 개 중 중복 선택 없이 r 개를 골라 순서 있게 나열하는 경우의 수의 일반식은? $1 \leq r \leq n$ 이다.

정답 ⑤ $n \times (n-1) \times \dots \times (n-r+1)$

해설

첫 자리에 n 가지, 다음에 $n-1$ 가지씩 r 개를 채우므로 n 부터 $n-r+1$ 까지 r 개의 수를 곱한다. ③은 조합의 수이고, ②는 인수를 하나 더 곱한 식이다.

문제 25 · 객관식

반복문으로 만들던 것과 동일한 후보를 모두 검사하도록 itertools로 코드를 바꾸었다. 이때 얻는 효과로 옳은 것은?

- ① 조건을 만족하지 않는 후보를 자동으로 건너뛴다
- ② 탐색해야 할 경우의 수가 줄어든다
- ③ 시간 복잡도의 차수가 한 단계 낮아진다
- ④ 중첩 반복문을 짧은 코드로 대체할 수 있다
- ⑤ 메모리를 전혀 사용하지 않고 동작한다

정답 및 해설 25

반복문으로 만들던 것과 동일한 후보를 모두 검사하도록 itertools로 코드를 바꾸었다. 이때 얻는 효과로 옳은 것은?

정답 ④ 중첩 반복문을 짧은 코드로 대체할 수 있다

해설

같은 후보를 모두 만들고 검사한다는 조건이므로 후보 수 자체는 줄지 않는다. permutations나 combinations를 쓰면 후보 생성 부분을 짧게 표현할 수 있다.

문제 26 · 객관식

숫자 1, 2, 3을 한 번씩 모두 사용해 만들 수 있는 세 자리 수의 개수는?

① 3개

② 6개

③ 9개

④ 27개

⑤ 12개

정답 및 해설 26

숫자 1, 2, 3을 한 번씩 모두 사용해 만들 수 있는 세 자리 수의 개수는?

정답 ② 6개

해설

세 개를 모두 사용해 순서 있게 나열하므로 $3! = 3 \times 2 \times 1 = 6$ 개다. `permutations(nums, 3)`으로 생성되는 후보의 개수와 같다.

문제 27 · 객관식

4가지 맛 중 서로 다른 2가지 맛을 골라 더블 컵을 만든다. 담은 순서를 구별하지 않을 때 가능한 메뉴는 몇 가지인가?

- ① 6가지
- ② 12가지
- ③ 8가지
- ④ 16가지
- ⑤ 4가지

정답 및 해설 27

4가지 맛 중 서로 다른 2가지 맛을 골라 더블 컵을 만든다. 담는 순서를 구별하지 않을 때 가능한 메뉴는 몇 가지인가?

정답 ① 6가지

해설

서로 다른 두 맛을 고르므로 $4 \times 3 \div 2 = 6$ 가지다. 같은 맛을 두 번 고르는 경우는 제외한다.

문제 28 · 객관식

1~45의 서로 다른 번호 중 6개를 중복 없이 고른다. 고른 순서를 구별하지 않을 때 경우의 수를 구하는 식은?

- ① $45! / 39!$
- ② $45! / (6! \times 39!)$
- ③ 45^6
- ④ $45 \times 44 \times 43 \times 42 \times 41 \times 40$
- ⑤ $6! / 45!$

정답 및 해설 28

1~45의 서로 다른 번호 중 6개를 중복 없이 고른다. 고른 순서를 구별하지 않을 때 경우의 수를 구하는 식은?

정답 ② $45! / (6! \times 39!)$

해설

조합의 개수는 $n! / (r! \times (n-r)!)$ 이므로 $45! / (6! \times 39!) = 8,145,060$ 이다. 네 번째 식은 순서를 구별한 $45P6$ 으로 이 값의 720배다.

문제 29 · 객관식

서로 다른 n 개 원소를 모두 나열하는 순열의 개수가 $n!$ 이라는 것이 의미하는 바로 가장 적절한 것은?

- ① 후보의 수가 n 의 값과 무관하게 항상 일정하게 유지된다
- ② n 이 아무리 커져도 후보의 수가 완만하게만 늘어난다
- ③ 후보의 수가 n 의 크기에 정확히 정비례해서 늘어난다
- ④ 후보의 수가 n 의 제곱에 비례하는 속도로 늘어난다
- ⑤ n 이 조금만 커져도 후보의 수가 감당할 수 없이 늘어난다

정답 및 해설 29

서로 다른 n 개 원소를 모두 나열하는 순열의 개수가 $n!$ 이라는 것이 의미하는 바로 가장 적절한 것은?

정답 ⑤ n 이 조금만 커져도 후보의 수가 감당할 수 없이 늘어난다

해설

$10! = 3,628,800$ 이지만 $20!$ 은 약 2.43×10^{18} 이다. 한 원소가 추가될 때마다 후보 수가 다음 정수 배로 늘어난다. 실제로 가능한 입력 크기는 후보당 작업량과 실행 환경에 따라 다르다.

문제 30 · 객관식

0~9의 서로 다른 숫자 3개를 순서 있게 나열한 비밀번호는 몇 개인가? 앞자리 0도 허용한다.

- ① 504개
- ② 1,000개
- ③ 120개
- ④ 30개
- ⑤ 720개

정답 및 해설 30

0~9의 서로 다른 숫자 3개를 순서 있게 나열한 비밀번호는 몇 개인가? 앞자리 0도 허용한다.

정답 ⑤ 720개

해설

서로 다른 세 숫자를 순서 있게 배치하므로 $10P3 = 10 \times 9 \times 8 = 720$ 개다. 중복을 허용했다면 $10^3 = 1,000$ 개가 된다.

문제 31 · 객관식

길이 n 인 텍스트에서 길이 m 인 패턴을 찾는다. $n \geq m \geq 1$ 일 때 패턴 전체가 들어갈 수 있는 시작 위치는 몇 개인가?

- ① $n + m - 1$ 개
- ② $n \times m - 1$ 개
- ③ $n - m + 1$ 개
- ④ $n - m - 1$ 개
- ⑤ $n \times (m - 1)$ 개

정답 및 해설 31

길이 n 인 텍스트에서 길이 m 인 패턴을 찾는다. $n \geq m \geq 1$ 일 때 패턴 전체가 들어갈 수 있는 시작 위치는 몇 개인가?

정답 ③ $n - m + 1$ 개

해설

패턴이 텍스트를 벗어나지 않으려면 시작 위치가 0부터 $n-m$ 까지여야 하므로 $n-m+1$ 개다. $n=11$, $m=3$ 이면 9개의 시작 위치를 검사한다.

문제 32 · 객관식

다음 함수에서 `bf_match("ababcabcbababd", "abc")`의 반환값은? 인덱스는 0부터 센다.

code.py

```
1 def bf_match(text, pattern):
2     n, m = len(text), len(pattern)
3     found = []
4     for i in range(n - m + 1):
5         ok = True
6         for j in range(m):
7             if text[i + j] != pattern[j]:
8                 ok = False; break
9         if ok: found.append(i)
10    return found
```

① [5, 8]

② [0, 2, 5]

③ [2]

④ [2, 5]

⑤ []

정답 및 해설 32

다음 함수에서 `bf_match("ababcabcbababd", "abc")`의 반환값은? 인덱스는 0부터 센다.

code.py

```
1 def bf_match(text, pattern):
2     n, m = len(text), len(pattern)
3     found = []
4     for i in range(n - m + 1):
5         ok = True
6         for j in range(m):
7             if text[i + j] != pattern[j]:
8                 ok = False; break
9         if ok: found.append(i)
10    return found
```

정답 ④ [2, 5]

해설

인덱스 2에서 abc, 인덱스 5에서 abc가 일치한다. 인덱스 0은 aba이므로 세 번째 문자에서 어긋난다.

문제 33 · 객관식

길이 n 인 문자열에서 길이 m 인 패턴을 찾는다($n \geq m \geq 1$). 시작 위치를 한 칸씩 옮기며 패턴의 왼쪽 글자부터 비교한다. 문자 비교가 $O(1)$ 일 때 최악 시간의 상한으로 가장 알맞은 것은?

- ① $O(n / m)$
- ② $O(n + m)$
- ③ $O(n)$
- ④ $O(m)$
- ⑤ $O(n \times m)$

정답 및 해설 33

길이 n 인 문자열에서 길이 m 인 패턴을 찾는다($n \geq m \geq 1$). 시작 위치를 한 칸씩 옮기며 패턴의 왼쪽 글자부터 비교한다. 문자 비교가 $O(1)$ 일 때 최악 시간의 상한으로 가장 알맞은 것은?

정답 ⑤ $O(n \times m)$

해설

시작 위치가 $n - m + 1$ 개이고 각 위치에서 최대 m 글자를 비교한다. 따라서 비교 횟수는 최대 $(n - m + 1)m$ 이며 시간 상한은 $O(nm)$ 이다.

문제 34 · 객관식

다음 코드에서 break 문이 하는 역할로 가장 적절한 것은?

code.py

```
1 def bf_match(text, pattern):
2     n, m = len(text), len(pattern)
3     found = []
4     for i in range(n - m + 1):
5         ok = True
6         for j in range(m):
7             if text[i + j] != pattern[j]:
8                 ok = False; break
9         if ok: found.append(i)
10    return found
```

- ① 한 글자라도 어긋나면 남은 비교를 생략하고 다음 시작 위치로 넘어간다
- ② 패턴을 찾으면 함수 전체를 즉시 끝내고 지금까지의 결과를 반환한다
- ③ 패턴 길이가 텍스트보다 길 때 발생하는 인덱스 오류를 막아 준다
- ④ 텍스트의 마지막 글자에 도달하면 바깥 반복문까지 함께 멈춘다
- ⑤ 이미 찾은 시작 위치를 다시 검사하지 않도록 건너뛰게 한다

정답 및 해설 34

다음 코드에서 break 문이 하는 역할로 가장 적절한 것은?

code.py

```
1 def bf_match(text, pattern):
2     n, m = len(text), len(pattern)
3     found = []
4     for i in range(n - m + 1):
5         ok = True
6         for j in range(m):
7             if text[i + j] != pattern[j]:
8                 ok = False; break
9         if ok: found.append(i)
10    return found
```

정답 ① 한 글자라도 어긋나면 남은 비교를 생략하고 다음 시작 위치로 넘어간다

해설

안쪽 반복문의 break이므로 그 시작 위치에 대한 비교만 중단된다. 한 글자만 달라도 그 위치는 이미 실패이므로 나머지를 볼 필요가 없다. 함수를 끝내는 것은 return이지 break가 아니다.

문제 35 · 객관식

다음 함수에서 `closest_pair([10, 3, 22, 15, 17, 9])`를 호출한다. 반환값 (`best`, `min_diff`) 중 `min_diff`는?

code.py

```
1 def closest_pair(arr):
2     min_diff = float('inf')
3     best = None
4     for i in range(len(arr) - 1):
5         for j in range(i + 1, len(arr)):
6             diff = abs(arr[i] - arr[j])
7             if diff < min_diff:
8                 min_diff = diff
9                 best = (arr[i], arr[j])
10    return best, min_diff
```

① 7

② 2

③ 3

④ 5

⑤ 1

정답 및 해설 35

다음 함수에서 `closest_pair([10, 3, 22, 15, 17, 9])`를 호출한다. 반환값 (`best`, `min_diff`) 중 `min_diff`는?

code.py

```
1 def closest_pair(arr):
2     min_diff = float('inf')
3     best = None
4     for i in range(len(arr) - 1):
5         for j in range(i + 1, len(arr)):
6             diff = abs(arr[i] - arr[j])
7             if diff < min_diff:
8                 min_diff = diff
9                 best = (arr[i], arr[j])
10    return best, min_diff
```

정답 ⑤ 1

해설

10과 9의 차이가 1로 가장 작으므로 `best=(10, 9)`, `min_diff=1`이다. 15와 17의 차이는 2이다. 이 코드는 모든 $i < j$ 쌍을 검사하므로 길이 n 에 대해 $O(n^2)$ 이다.

문제 36 · 객관식

arr에는 정수가 2개 이상 있다. float('inf')는 모든 정수보다 큰 양의 무한대이다. 다음 코드에서 min_diff를 이 값으로 시작하는 이유는?

code.py

```
1 def closest_pair(arr):
2     min_diff = float('inf')
3     best = None
4     for i in range(len(arr) - 1):
5         for j in range(i + 1, len(arr)):
6             diff = abs(arr[i] - arr[j])
7             if diff < min_diff:
8                 min_diff = diff
9                 best = (arr[i], arr[j])
10    return best, min_diff
```

- ① 정렬되지 않은 리스트도 처리할 수 있게 하기 위해
- ② 차이를 정수가 아닌 실수형으로 계산하도록 하기 위해
- ③ 두 수의 차이가 음수가 되는 경우를 미리 막기 위해
- ④ 리스트가 비어 있을 때 생기는 오류를 피하기 위해
- ⑤ 첫 번째로 계산되는 차이가 반드시 갱신되도록 하기 위해

정답 및 해설 36

arr에는 정수가 2개 이상 있다. float('inf')는 모든 정수보다 큰 양의 무한대이다. 다음 코드에서 min_diff를 이 값으로 시작하는 이유는?

code.py

```
1 def closest_pair(arr):
2     min_diff = float('inf')
3     best = None
4     for i in range(len(arr) - 1):
5         for j in range(i + 1, len(arr)):
6             diff = abs(arr[i] - arr[j])
7             if diff < min_diff:
8                 min_diff = diff
9                 best = (arr[i], arr[j])
10    return best, min_diff
```

정답 ⑤ 첫 번째로 계산되는 차이가 반드시 갱신되도록 하기 위해

해설

0으로 초기화하면 어떤 차이도 그보다 작지 않아 갱신이 일어나지 않는다. 최소값을 구할 때는 가능한 어떤 값보다 큰 값으로, 최대값을 구할 때는 반대로 초기화한다.

문제 37 · 객관식

원소가 n 개인 집합의 부분집합 개수는? (공집합과 자기 자신 포함)

① n^2 개

② 2^n 개

③ $n!$ 개

④ $2n$ 개

⑤ $n(n-1)/2$ 개

정답 및 해설 37

원소가 n 개인 집합의 부분집합 개수는? (공집합과 자기 자신 포함)

정답 ② 2^n 개

해설

각 원소마다 포함하거나 포함하지 않는 두 가지 선택이 있으므로 2를 n 번 곱한다. 원소 3개면 8개, 10개면 1,024개, 20개면 약 100만 개다.

문제 38 · 객관식

다음 코드에서 `arr = ['A', 'B', 'C']` 이고 `i = 5` 일 때 만들어지는 부분집합은?

code.py

```
1 def power_set(arr):
2     n = len(arr)
3     subsets = []
4     for i in range(1 << n):
5         cur = []
6         for j in range(n):
7             if i & (1 << j):
8                 cur.append(arr[j])
9         subsets.append(cur)
10    return subsets
```

① ['A', 'B', 'C']

② ['A', 'B']

③ ['B', 'C']

④ ['A', 'C']

⑤ ['C']

정답 및 해설 38

다음 코드에서 `arr = ['A', 'B', 'C']` 이고 `i = 5` 일 때 만들어지는 부분집합은?

code.py

```
1 def power_set(arr):
2     n = len(arr)
3     subsets = []
4     for i in range(1 << n):
5         cur = []
6         for j in range(n):
7             if i & (1 << j):
8                 cur.append(arr[j])
9         subsets.append(cur)
10    return subsets
```

정답 ④ ['A', 'C']

해설

5를 이진수로 쓰면 101이다. 0번 비트와 2번 비트가 1이므로 `arr[0]`인 A와 `arr[2]`인 C가 선택된다.

문제 39 · 객관식

Python에서 n 이 0 이상의 정수일 때, 부분집합 생성에 쓰는 $1 \ll n$ 의 값은?

- ① $n / 2$
- ② n^2
- ③ 2^n
- ④ $2n$
- ⑤ $n!$

정답 및 해설 39

Python에서 n 이 0 이상의 정수일 때, 부분집합 생성에 쓰는 $1 \ll n$ 의 값은?

정답 ③ 2^n

해설

1을 왼쪽으로 n 비트 이동하면 2를 n 번 곱한 것과 같다. 따라서 $\text{range}(1 \ll n)$ 은 0부터 $2^n - 1$ 까지를 순회하며 모든 부분집합에 하나씩 대응한다.

문제 40 · 객관식

조합적 폭발(combinatorial explosion)을 가장 잘 설명한 것은?

- ① 반복문 안에서 예외가 연쇄적으로 발생해 프로그램이 멈추어 버리는 현상
- ② 입력이 커질수록 프로그램이 사용하는 메모리가 일정하게 늘어나는 현상
- ③ 확인해야 할 경우의 수가 입력 크기에 정비례해 완만하게 늘어나는 현상
- ④ 입력이 조금만 커져도 확인할 경우의 수가 감당할 수 없이 늘어나는 현상
- ⑤ 계산 결과의 자릿수가 넘쳐 값이 잘못 저장되어 버리는 현상

정답 및 해설 40

조합적 폭발(combinatorial explosion)을 가장 잘 설명한 것은?

정답 ④ 입력이 조금만 커져도 확인할 경우의 수가 감당할 수 없이 늘어나는
현상

해설

n 이 20 정도만 되어도 2^n 은 약 100만, $n!$ 은 약 2.4×10^{18} 이 된다. 완전 탐색이 실패하는 거의 모든 경우가 이 현상 때문이다.

문제 41 · 객관식

$n=20$ 일 때 값이 가장 큰 것은? $\log n$ 은 밑이 2인 로그를 뜻한다.

- ① 2^n
- ② $n!$
- ③ n^2
- ④ n^3
- ⑤ $n \log n$

정답 및 해설 41

$n=20$ 일 때 값이 가장 큰 것은? $\log n$ 은 밑이 2인 로그를 뜻한다.

정답 ② $n!$

해설

$20!$ 은 약 2.43×10^{18} , 2^{20} 은 약 100만, 20^2 은 400, 20^3 은 8,000이다. 팩토리얼이 압도적으로 크며 이 장에서 다루는 세 가지 폭발 유형 중 가장 가파르다.

문제 42 · 객관식

k 는 입력 크기 n 과 무관한 고정 양의 정수이다. 다음 중 n 에 대한 다항식 증가를 나타내는 유형은?

- ① $O(2^n)$
- ② $O(n^k)$
- ③ $O(n!)$
- ④ $O(n^n)$
- ⑤ $O(\log n)$

정답 및 해설 42

k 는 입력 크기 n 과 무관한 고정 양의 정수이다. 다음 중 n 에 대한 다항식 증가를 나타내는 유형은?

정답 ② $O(n^k)$

해설

반복문을 k 겹 중첩하면 $O(n^k)$ 이 된다. 2^n 은 포함·비포함을 반복하는 선택 구조에서, $n!$ 은 순서를 나열하는 구조에서 나타난다.

문제 43 · 객관식

서로 다른 n 개 원소의 부분집합 후보 수가 2^n 개가 되는 근본적인 이유는?

- ① 부분집합의 크기가 원소마다 달라지기 때문이다
- ② 원소를 순서대로 나열하는 방법이 여러 가지이기 때문이다
- ③ 반복문을 n 겹으로 중첩해야 하기 때문이다
- ④ 각 원소를 다른 모든 원소와 한 번씩 비교하기 때문이다
- ⑤ 원소마다 포함할지 말지 두 갈래의 선택이 반복되기 때문이다

정답 및 해설 43

서로 다른 n 개 원소의 부분집합 후보 수가 2^n 개가 되는 근본적인 이유는?

정답 ⑤ 원소마다 포함할지 말지 두 갈래의 선택이 반복되기 때문이다

해설

원소 하나가 늘 때마다 경우의 수가 두 배가 된다. 시작이 1가지, 원소 1개면 2가지, 2개면 4가지, n 개면 2^n 가지가 되는 구조다.

문제 44 · 객관식

서로 다른 원소 10개를 모두 나열하는 순열은 $10!$ 개다. 원소가 11개가 되면 순열 수는 몇 배가 되는가?

- ① 2배
- ② 11배
- ③ 10배
- ④ 100배
- ⑤ 121배

정답 및 해설 44

서로 다른 원소 10개를 모두 나열하는 순열은 $10!$ 개다. 원소가 11개가 되면 순열 수는 몇 배가 되는가?

정답 ② 11배

해설

$11! = 11 \times 10!$ 이므로 11배다. $10!$ 의 십진수 값을 외우거나 계산할 필요는 없다.

문제 45 · 객관식

비밀번호의 각 자리에 0~9를 중복해서 쓸 수 있고 앞자리 0도 허용한다. 자릿수를 하나 늘리면 후보 수는 어떻게 변하는가?

- ① 2배가 된다
- ② 10배가 된다
- ③ 10이 더해진다
- ④ 제곱이 된다
- ⑤ 변하지 않는다

정답 및 해설 45

비밀번호의 각 자리에 0~9를 중복해서 쓸 수 있고 앞자리 0도 허용한다. 자릿수를 하나 늘리면 후보 수는 어떻게 변하는가?

정답 ② 10배가 된다

해설

자리마다 10가지 선택이 곱해지므로 자릿수가 하나 늘 때마다 10배가 된다. 4자리 1만, 6자리 100만, 8자리 1억으로 늘어난다.

문제 46 · 객관식

n 개 도시($n \geq 2$) 중 출발 도시를 고정하고, 나머지를 한 번씩 방문한 뒤 돌아온다. 역순도 별개로 세어 모든 방문 순서를 검사한다. 순서마다 n 개 이동 비용을 더할 때 전체 시간 복잡도는?

- ① $O(n^2)$
- ② $O(2^n)$
- ③ $O(n!)$
- ④ $O(n^3)$
- ⑤ $O(n \log n)$

정답 및 해설 46

n 개 도시($n \geq 2$) 중 출발 도시를 고정하고, 나머지를 한 번씩 방문한 뒤 돌아온다. 역순도 별개로 세어 모든 방문 순서를 검사한다. 순서마다 n 개 이동 비용을 더할 때 전체 시간 복잡도는?

정답 ③ $O(n!)$

해설

출발 도시를 고정하면 나머지 $n-1$ 개 도시의 순서가 $(n-1)!$ 개다. 각 순회의 이동 비용을 n 번 더하므로 전체 시간은 $O(n \cdot (n-1)!) = O(n!)$ 이다.

문제 47 · 객관식

서로 다른 도시 4개의 방문 순서를 모두 나열한다. 출발 도시는 고정하지 않으며, 역순도 다른 순서로 센다. 가능한 순서는 몇 가지인가?

- ① 64가지
- ② 16가지
- ③ 12가지
- ④ 24가지
- ⑤ 4가지

정답 및 해설 47

서로 다른 도시 4개의 방문 순서를 모두 나열한다. 출발 도시는 고정하지 않으며, 역순도 다른 순서로 센다. 가능한 순서는 몇 가지인가?

정답 ④ 24가지

해설

첫 도시 4가지, 다음 3가지, 다음 2가지, 마지막 1가지이므로 $4!=24$ 가지다.

문제 48 · 객관식

서로 다른 20개 원소의 순열은 $20! \approx 2.43 \times 10^{18}$ 개이다. 초당 1억 개의 순열을 검사할 수 있어도 모든 순열을 검사하기 어려운 이유는?

- ① 메모리 용량이 순열 하나를 담기에 부족하기 때문이다
- ② 초당 1억 개를 검사해도 수백 년이 걸리기 때문이다
- ③ 파이썬의 재귀 깊이 제한에 걸리기 때문이다
- ④ 정수 자료형의 범위를 넘어서기 때문이다
- ⑤ 순열을 생성하는 함수가 20개까지만 지원하기 때문이다

정답 및 해설 48

서로 다른 20개 원소의 순열은 $20! \approx 2.43 \times 10^{18}$ 개이다. 초당 1억 개의 순열을 검사할 수 있어도 모든 순열을 검사하기 어려운 이유는?

정답 ② 초당 1억 개를 검사해도 수백 년이 걸리기 때문이다

해설

$20!$ 은 약 2.43×10^{18} 이고, 초당 1억 번을 처리해도 약 770년이 걸린다. 메모리나 자료형이 아니라 계산량 자체가 문제다.

문제 49 · 객관식

완전 탐색이 제한 시간 안에 끝날지 추정하려고 한다. 총실행량을 계산하는 데 직접 필요한 정보는?

- ① 후보 수와 후보 하나를 검사하는 데 드는 작업량
- ② 사용할 반복문의 개수를 정하는 일
- ③ 입력 데이터가 정렬되어 있는지 확인하는 일
- ④ 재귀와 반복 중 어느 쪽으로 구현할지 정하는 일
- ⑤ 출력 형식이 무엇인지 확인하는 일

정답 및 해설 49

완전 탐색이 제한 시간 안에 끝날지 추정하려고 한다. 총실행량을 계산하는 데 직접 필요한 정보는?

정답 ① 후보 수와 후보 하나를 검사하는 데 드는 작업량

해설

후보가 100만 개라도 하나를 한 번 비교하는 경우와 길이 1만인 배열을 읽는 경우의 실행량은 다르다. 후보 수×후보당 작업량으로 추정한 뒤 실행 환경과 제한 시간을 확인한다.

문제 50 · 객관식

완전 탐색에 가지치기(pruning)를 더한 기법은?

- ① 동적 계획법
- ② 분할 정복
- ③ 백트래킹
- ④ 그리디 알고리즘
- ⑤ 이진 탐색

정답 및 해설 50

완전 탐색에 가지치기(pruning)를 더한 기법은?

정답 ③ 백트래킹

해설

정답이 될 수 없다고 판단되는 가지를 미리 잘라 내어 탐색량을 줄이는 것이 백트래킹이다. N-Queens와 스도쿠가 대표적인 적용 사례다.

문제 51 · 객관식

완전 탐색에 기억(메모이제이션)을 더한 기법은?

- ① 완전 탐색
- ② 백트래킹
- ③ 분할 정복
- ④ 동적 계획법
- ⑤ 선형 탐색

정답 및 해설 51

완전 탐색에 기억(메모이제이션)을 더한 기법은?

정답 ④ 동적 계획법

해설

한 번 계산한 부분 문제의 결과를 저장해 두었다가 재사용한다. 피보나치 수열이 완전 탐색으로는 $O(2^n)$ 이지만 메모이제이션을 쓰면 $O(n)$ 이 되는 것이 대표적인 예다.

문제 52 · 객관식

양의 정수들에서 합이 10인 부분집합을 찾는다. 원소를 하나씩 고를 때 백트래킹으로 불필요한 탐색을 줄일 수 있는 이유는?

- ① 더하다가 이미 10을 넘으면 그 갈래를 더 이상 보지 않기 때문이다
- ② 부분집합을 만들지 않고 원소의 합만 따로 계산해서 비교하기 때문이다
- ③ 한 번 계산해 둔 합을 저장했다가 필요할 때 그대로 다시 쓰기 때문이다
- ④ 부분집합을 크기가 작은 것부터 차례대로 만들어 보고 멈추기 때문이다
- ⑤ 양수인 원소를 더 고르면 현재 합이 줄어들 수 있기 때문이다

정답 및 해설 52

양의 정수들에서 합이 10인 부분집합을 찾는다. 원소를 하나씩 고를 때 백트래킹으로 불필요한 탐색을 줄일 수 있는 이유는?

정답 ① 더하다가 이미 10을 넘으면 그 갈래를 더 이상 보지 않기 때문이다

해설

원소가 모두 양수라서 합이 10을 넘으면 원소를 더 골라도 10으로 줄어들지 않는다. 따라서 그 갈래를 중단할 수 있다.

문제 53 · 객관식

n 은 0 이상의 정수이다. $F(0)=0$, $F(1)=1$, $n \geq 2$ 에서 $F(n)=F(n-1)+F(n-2)$ 이다. 아래 코드는 같은 $F(k)$ 를 매번 다시 계산하는 단순 재귀와 비교해 어떤 중복 작업을 줄이는가?

code.py

```
1 memo = {}
2 def fib(n):
3     if n in memo:
4         return memo[n]
5     if n <= 1:
6         return n
7     memo[n] = fib(n-1) + fib(n-2)
8     return memo[n]
```

- ① 딕셔너리 대신 곱셈을 사용해 계산량을 줄였기 때문이다
- ② 계산 도중 가망이 없는 값을 미리 걸러 내고 넘어가기 때문이다
- ③ 재귀 호출을 모두 반복문으로 바꾸어 구현했기 때문이다
- ④ 문제를 절반씩 나누어 따로 계산한 뒤 합치기 때문이다
- ⑤ 이미 계산한 값을 저장해 두고 다시 계산하지 않기 때문이다

정답 및 해설 53

n 은 0 이상의 정수이다. $F(0)=0$, $F(1)=1$, $n \geq 2$ 에서 $F(n)=F(n-1)+F(n-2)$ 이다. 아래 코드는 같은 $F(k)$ 를 매번 다시 계산하는 단순 재귀와 비교해 어떤 중복 작업을 줄이는가?

code.py

```
1 memo = {}
2 def fib(n):
3     if n in memo:
4         return memo[n]
5     if n <= 1:
6         return n
7     memo[n] = fib(n-1) + fib(n-2)
8     return memo[n]
```

정답 ⑤ 이미 계산한 값을 저장해 두고 다시 계산하지 않기 때문이다

해설

예를 들어 $F(4)$ 를 구할 때 $F(2)$ 가 여러 번 필요하다. 이 코드는 첫 계산 결과를 $memo[2]$ 에 저장하고 다음에는 바로 읽는다. 덧셈과 기록 조회를 $O(1)$ 로 보면, 단순 재귀의 시간 상한 $O(2^n)$ 을 $O(n)$ 으로 줄인다.

문제 54 · 객관식

n 개 항목을 이름순으로 정렬한 전화번호부에서 이름을 찾는다. 이름 비교를 $O(1)$ 로 볼 때, 완전 탐색 방식과 가장 정확한 최악 시간 복잡도를 짝지은 것은?

- ① 첫 페이지부터 한 줄씩 확인 — $O(\log n)$
- ② 중간을 펼쳐 절반씩 버림 — $O(\log n)$
- ③ 첫 페이지부터 한 줄씩 확인 — $O(n)$
- ④ 중간을 펼쳐 절반씩 버림 — $O(n)$
- ⑤ 첫 페이지부터 한 줄씩 확인 — $O(n^2)$

정답 및 해설 54

n 개 항목을 이름순으로 정렬한 전화번호부에서 이름을 찾는다. 이름 비교를 $O(1)$ 로 볼 때, 완전 탐색 방식과 가장 정확한 최악 시간 복잡도를 짝지은 것은?

정답 ③ 첫 페이지부터 한 줄씩 확인 — $O(n)$

해설

완전 탐색에 해당하는 것은 처음부터 끝까지 훑는 선형 검색으로 $O(n)$ 이다. 절반씩 버리고 한쪽만 남기는 이진 탐색은 축소 정복이며 $O(\log n)$ 이다.

문제 55 · 객관식

알고리즘 설계에서 완전 탐색을 먼저 구현해 보라고 권하는 이유로 가장 적절한 것은?

- ① 완전 탐색이 다른 어떤 기법보다 메모리를 적게 쓰기 때문이다
- ② 완전 탐색은 입력 크기와 무관하게 항상 같은 속도로 동작하기 때문이다
- ③ 완전 탐색은 코드가 길어져서 오히려 실수를 줄여 주기 때문이다
- ④ 완전 탐색이 대부분의 문제에서 가장 빠른 방법이기 때문이다
- ⑤ 문제의 구조를 파악하고 정답을 확인할 기준을 얻을 수 있기 때문이다

정답 및 해설 55

알고리즘 설계에서 완전 탐색을 먼저 구현해 보라고 권하는 이유로 가장 적절한 것은?

정답 ⑤ 문제의 구조를 파악하고 정답을 확인할 기준을 얻을 수 있기 때문이다

해설

느리더라도 확실히 맞는 답을 주므로, 최적화한 코드의 결과를 검증하는 기준으로 쓸 수 있고 문제의 구조도 드러난다. 그다음에 어디를 줄일지 고민하는 것이 올바른 순서다.

문제 56 · 객관식

코딩 테스트에서 입력이 많을 때 `input()` 대신 `sys.stdin.readline` 을 쓰는 이유는?

- ① 잘못된 입력이 들어오면 예외를 발생시키기 때문이다
- ② 입력값의 자료형을 자동으로 변환해 주기 때문이다
- ③ 줄 끝의 개행 문자를 자동으로 없애 주기 때문이다
- ④ 여러 줄을 한 번에 리스트로 만들어 주기 때문이다
- ⑤ 대량 입력을 보통 더 빠르게 읽을 수 있기 때문이다

정답 및 해설 56

코딩 테스트에서 입력이 많을 때 `input()` 대신 `sys.stdin.readline` 을 쓰는 이유는?

정답 ⑤ 대량 입력을 보통 더 빠르게 읽을 수 있기 때문이다

해설

`sys.stdin.readline`은 대량 입력에서 보통 오버헤드가 적다. 줄 끝 개행을 그대로 남기므로 개행만 제거하려면 `rstrip('\n')`을 쓴다. `strip()`은 앞뒤 공백까지 제거한다.

문제 57 · 객관식

다음 코드의 출력값은?

code.py

```
1 from itertools import combinations
2 arr = [1, 2, 3, 4, 5]
3 target = 9
4 count = 0
5 for c in combinations(arr, 3):
6     if sum(c) == target:
7         count += 1
8 print(count)
```

① 5

② 1

③ 3

④ 2

⑤ 10

정답 및 해설 57

다음 코드의 출력값은?

code.py

```
1 from itertools import combinations
2 arr = [1, 2, 3, 4, 5]
3 target = 9
4 count = 0
5 for c in combinations(arr, 3):
6     if sum(c) == target:
7         count += 1
8 print(count)
```

정답 ④ 2

해설

1부터 5까지에서 서로 다른 세 수를 골라 합이 9가 되는 경우는 (1,3,5)와 (2,3,4) 두 가지다. combinations는 순서를 구별하지 않으므로 (3,1,5) 같은 것은 따로 세지 않는다.

문제 58 · 객관식

다음 코드의 출력값은?

code.py

```
1 from itertools import combinations
2 def blackjack(cards, M):
3     best = 0
4     for c in combinations(cards, 3):
5         t = sum(c)
6         if t <= M and t > best:
7             best = t
8     return best
9 print(blackjack([5, 6, 7, 8, 9], 21))
```

① 22

② 20

③ 18

④ 24

⑤ 21

정답 및 해설 58

다음 코드의 출력값은?

code.py

```
1 from itertools import combinations
2 def blackjack(cards, M):
3     best = 0
4     for c in combinations(cards, 3):
5         t = sum(c)
6         if t <= M and t > best:
7             best = t
8     return best
9 print(blackjack([5, 6, 7, 8, 9], 21))
```

정답 ⑤ 21

해설

세 장의 합이 21을 넘지 않으면서 가장 큰 경우를 찾는다. $5+7+9 = 21$ 이 조건을 만족하는 최대값이다. $6+8+9 = 23$ 은 21을 넘으므로 제외된다.

문제 59 · 객관식

서로 다른 원소 6개에서 순서 없이 3개를 고른다. 같은 원소는 두 번 고르지 않는다. 확인할 조합은 몇 가지인가?

- ① 6가지
- ② 18가지
- ③ 120가지
- ④ 20가지
- ⑤ 216가지

정답 및 해설 59

서로 다른 원소 6개에서 순서 없이 3개를 고른다. 같은 원소는 두 번 고르지 않는다. 확인할 조합은 몇 가지인가?

정답 ④ 20가지

해설

$6 \times 5 \times 4$ 를 같은 세 원소의 나열 순서 $3 \times 2 \times 1$ 로 나누면 20가지다.

문제 60 · 객관식

다음 코드에서 회문인지 판단하는 `s[::-1]`의 의미는?

code.py

```
1 s = input().strip()
2 if s == s[::-1]:
3     print('회문')
4 else:
5     print('아님')
```

- ① 문자열에서 첫 글자를 제외한 나머지
- ② 문자열의 마지막 한 글자
- ③ 문자열을 거꾸로 뒤집은 결과
- ④ 문자열을 한 글자씩 나눈 리스트
- ⑤ 문자열의 길이에서 1을 뺀 값

정답 및 해설 60

다음 코드에서 회문인지 판단하는 `s[::-1]`의 의미는?

code.py

```
1 s = input().strip()
2 if s == s[::-1]:
3     print('회문')
4 else:
5     print('아님')
```

**정답 ③ 문자열을 거꾸로 뒤
집은 결과**

해설

슬라이싱의 세 번째 값이 `-1`이면 뒤에서부터 한 칸씩 이동하며 읽으므로 문자열이 뒤집힌다. 뒤집은 결과가 원래와 같으면 앞뒤로 읽어도 같은 회문이다.

네 자리 자물쇠 탐색

주어진 것: 숫자 0~9로 된 정확히 네 자리 문자열 secret. 앞자리 0도 가능하다.

할 일: 0000, 0001, ..., 9999 순서로 네 겹 반복문을 돌려 secret을 찾는다.

반환할 것: secret을 찾을 때까지의 시도 횟수. 0000이 첫 번째 시도이다.

입력 `lock_attempts("8234")`

결과 8235

작성할 내용

Python과 C 모두 시도 횟수를 반환한다.

예: "0000"이면 1, "0001"이면 2이다.

자릿수가 네 자리로 고정된 점을 반영해 시간 복잡도를 주석으로 쓰시오.

Python: `lock_attempts(secret)`

C: `int lock_attempts(const char secret[])`

네 자리 자물쇠 탐색

```
1 def lock_attempts(secret):
2     count = 0
3     for a in range(10):
4         for b in range(10):
5             for c in range(10):
6                 for d in range(10):
7                     count += 1
8                     if f"{a}{b}{c}{d}" == secret:
9                         return count
```

시간 복잡도: 자릿수 4가 고정이므로 최대 10,000번 검사, $O(1)$.

네 자리 자물쇠 탐색

필요한 표준 헤더: <stdlib.h> <string.h> <limits.h> / 결과 배열은 호출자가 준비

```
1 int lock_attempts(const char secret[]) {
2     int count = 0;
3     for (int a = 0; a < 10; a++)
4         for (int b = 0; b < 10; b++)
5             for (int c = 0; c < 10; c++)
6                 for (int d = 0; d < 10; d++) {
7                     char attempt[5] = {'0'+a, '0'+b, '0'+c, '0'+d, '\0'};
8                     count++;
9                     if (strcmp(attempt, secret) == 0) return count;
10                }
11    return 0;
12 }
```

시간 복잡도: 자릿수 4가 고정이므로 최대 10,000번 검사, $O(1)$.

개념 및 계산

0~9로 이루어진 4자리 자물쇠를 0000부터 순서대로 모두 시도할 때, 마지막까지 갔다면 총 몇 회 시도한 것인지 쓰시오.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

0~9로 이루어진 4자리 자물쇠를 0000부터 순서대로 모두 시도할 때, 마지막까지 갔다면 총 몇 회 시도한 것인지 쓰시오.

정답 10,000회

해설

0000부터 9999까지 빠짐없이 시도하면 $10^4 = 10,000$ 회다. 정답을 찾는 즉시 멈추지 않고 끝까지 돌린 경우의 횟수다.

개념 및 계산

0~9를 중복해서 쓰고 앞자리 0도 허용하는 4자리 자물쇠를 0000부터 한 번씩 시도한다. 초당 1,000회라면 최악의 소요 시간은?

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

0~9를 중복해서 쓰고 앞자리 0도 허용하는 4자리 자물쇠를 0000부터 한 번씩 시도한다. 초당 1,000회라면 최악의 소요 시간은?

정답 10초

해설

$10,000 \div 1,000 = 10$ 초다. 같은 속도로 8자리를 시도하면 $1억 \div 1,000 = 10만$ 초, 약 28시간이 걸린다.

개념 및 계산

완전 탐색에서 '탐색 공간'이라는 말이 가리키는 것을 한 문장으로 쓰시오.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

완전 탐색에서 '탐색 공간'이라는 말이 가리키는 것을 한 문장으로 쓰시오.

정답 알고리즘이 확인할 모든 후보의 집합

해설

실행 시간은 후보 수와 후보당 처리 비용에 좌우된다. 코드를 작성하기 전에 두 값을 함께 살펴본다.

합이 K인 두 수의 목록

주어진 것: 정수 배열 A(길이 n, $0 \leq n \leq 30$)와 target. 원소와 target은 $-100 \sim 100$ 이다.

반환할 것: $0 \leq i < j < n$ 이고 $A[i] + A[j] = \text{target}$ 인 값 쌍 $[A[i], A[j]]$ 전부.

i를 작은 순서로, 같은 i에서는 j를 작은 순서로 검사해 담는다. 같은 값도 위치가 다르면 별개의 선택이다.

입력 `find_pairs([1,3,5,7,9], 10)`

결과 `[[1,9],[3,7]]`

작성할 내용

Python: 값 쌍들의 목록, 없으면 [].

C: 준비된 out의 각 행에 쌍을 쓰고 행 수를 반환한다. 최대 435행이 준비되어 있다.

없으면 0.

시간 복잡도를 주석으로 쓰시오.

Python: `find_pairs(A, target)`

C: `int find_pairs(const int A[], int n, int target, int out[][2])`

합이 K인 두 수의 목록

```
1 def find_pairs(A, target):
2     result = []
3     for i in range(len(A)):
4         for j in range(i + 1, len(A)):
5             if A[i] + A[j] == target:
6                 result.append([A[i], A[j]])
7     return result
```

시간 $O(n^2)$.

합이 K인 두 수의 목록

필요한 표준 헤더: <stdlib.h> <string.h> <limits.h> / 결과 배열은 호출자가 준비

```
1 int find_pairs(const int A[], int n, int target, int out[][2]) {
2     int count = 0;
3     for (int i = 0; i < n; i++)
4         for (int j = i + 1; j < n; j++)
5             if (A[i] + A[j] == target) {
6                 out[count][0] = A[i]; out[count++][1] = A[j];
7             }
8     return count;
9 }
```

시간 $O(n^2)$.

개념 및 계산

눈이 1~6인 서로 구별되는 주사위 세 개를 던진다. 나온 눈의 합이 4인 경우의 수를 쓰시오.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

눈이 1~6인 서로 구별되는 주사위 세 개를 던진다. 나온 눈의 합이 4인 경우의 수를 쓰시오.

정답 3가지

해설

(1,1,2), (1,2,1), (2,1,1) 세 가지다. 삼중 반복문으로 216가지를 모두 만든 뒤 조건을 통과한 것만 세면 이 값이 나온다.

개념 및 계산

길이 1,000인 리스트에서 인덱스 $i < j$ 인 모든 쌍을 정확히 한 번씩 검사한다. 총 검사 횟수를 정확한 정수로 쓰시오.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

길이 1,000인 리스트에서 인덱스 $i < j$ 인 모든 쌍을 정확히 한 번씩 검사한다. 총 검사 횟수를 정확한 정수로 쓰시오.

정답 499,500회

해설

첫 위치에서 999쌍, 다음 위치에서 998쌍, 마지막에서 1쌍이다.
 $999 + 998 + \dots + 1 = 1,000 \times 999 / 2 = 499,500$ 이다.

개념 및 계산

$n \geq 2$ 이다. 아래 두 코드에서 check는 호출될 때마다 한 번 검사한다. 변경 후의 총 검사 횟수는 변경 전의 몇 배인가?

[변경 전]

```
for i in range(n - 1):  
    for j in range(i + 1, n): check(i, j)
```

[변경 후]

```
for i in range(n - 1):  
    for j in range(n): check(i, j)
```

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

$n \geq 2$ 이다. 아래 두 코드에서 check는 호출될 때마다 한 번 검사한다. 변경 후의 총 검사 횟수는 변경 전의 몇 배인가?

[변경 전]

```
for i in range(n - 1):  
    for j in range(i + 1, n): check(i, j)
```

[변경 후]

```
for i in range(n - 1):  
    for j in range(n): check(i, j)
```

정답 정확히 2배

해설

원래는 $n(n-1)/2$ 회, 변경 후에는 $(n-1)n$ 회이다. 바깥 반복문의 범위는 그대로이므로 비율은 정확히 2이다.

블랙잭 최적 조합

주어진 것: 카드 값 배열 A(길이 n, $0 \leq n \leq 100$)와 target. 카드 값은 1~1,000, target은 0~3,000이다.
할 일: 서로 다른 위치의 카드 세 장을 고른다. 값이 같은 카드도 고를 수 있다.
반환할 것: target 이하인 세 장의 합 중 가장 큰 값. 가능한 선택이 없으면 0이다.

입력 `blackjack([5,6,7,8,9], 21)`
결과 21

작성할 내용

Python과 C 모두 최대 합을 정수로 반환한다.
같은 위치의 카드를 두 번 고르면 안 된다.
시간 복잡도를 주석으로 쓰시오.

Python: `blackjack(A, target)`
C: `int blackjack(const int A[], int n, int target)`

블랙잭 최적 조합

```
1 def blackjack(A, target):
2     best = 0
3     for i in range(len(A)):
4         for j in range(i + 1, len(A)):
5             for k in range(j + 1, len(A)):
6                 total = A[i] + A[j] + A[k]
7                 if total <= target and total > best:
8                     best = total
9     return best
```

시간 $O(n^3)$.

블랙잭 최적 조합

필요한 표준 헤더: <stdlib.h> <string.h> <limits.h> / 결과 배열은 호출자가 준비

```
1 int blackjack(const int A[], int n, int target) {
2     int best=0;
3     for(int i=0;i<n;i++) for(int j=i+1;j<n;j++) for(int k=j+1;k<n;k++) {
4         int total=A[i]+A[j]+A[k];
5         if(total<=target && total>best) best=total;
6     }
7     return best;
8 }
```

시간 $O(n^3)$.

개념 및 계산

다음 후보를 만드는 도구를 각각 고르시오($1 \leq r \leq n$).

보기: for 반복문 / permutations / combinations

- ① 1부터 n 까지 정수를 하나씩 확인
- ② 서로 다른 n 개 중 r 개를 순서 있게 나열
- ③ 서로 다른 n 개 중 r 개를 순서 없이 선택

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

다음 후보를 만드는 도구를 각각 고르시오($1 \leq r \leq n$).

보기: for 반복문 / permutations / combinations

- ① 1부터 n 까지 정수를 하나씩 확인
- ② 서로 다른 n 개 중 r 개를 순서 있게 나열
- ③ 서로 다른 n 개 중 r 개를 순서 없이 선택

정답 ① for 반복문 ② permutations ③ combinations

해설

①은 수를 하나씩 순회한다. ②는 같은 원소라도 나열 순서가 다르면 다른 결과다. ③은 순서가 달라도 같은 선택으로 취급한다.

r개 순열 직접 생성

주어진 것: 서로 다른 정수 배열 A(길이 n)와 선택 개수 r. $0 \leq r \leq n \leq 6$ 이다.

반환할 것: 같은 원소를 두 번 쓰지 않고 r개를 순서 있게 나열한 모든 결과.

첫 칸을 A의 앞에서부터 고르고, 다음 칸도 아직 안 쓴 원소를 앞에서부터 고른다. r=0이면 빈 결과 하나이다. 순열 라이브러리는 쓰지 않는다.

```
make_permutations([3,1,2], 2)
→ [[3,1],[3,2],[1,3],[1,2],[2,3],[2,1]]
```

작성할 내용

Python: 결과 목록, r=0이면 [[]].

C: out에 한 결과씩 r칸을 쓰고 행 수 반환.

720행이 준비되어 있다. r=0이면 1 반환.

시간 복잡도를 주석으로 쓰시오.

Python: `make_permutations(A, r)`

C: `int make_permutations(const int A[], int n, int r, int out[][6])`

r개 순열 직접 생성

```
1 def make_permutations(A, r):
2     result, path = [], []
3     used = [False] * len(A)
4     def dfs():
5         if len(path) == r:
6             result.append(path.copy())
7             return
8         for i in range(len(A)):
9             if not used[i]:
10                used[i] = True
11                path.append(A[i])
12                dfs()
13                path.pop()
14                used[i] = False
15     dfs()
16     return result
```

시간 상한 $O(n \cdot n!)$, 결과 수 nPr . 각 결과를 기록할 때 r 개 원소를 복사한다.

r개 순열 직접 생성

필요한 표준 헤더: <stdlib.h> <string.h> <limits.h> / 결과 배열은 호출자가 준비

```

1 static void perm_visit(const int A[], int n, int r, int depth, int used[], int path[],
2     int out[][6], int *count) {
3     if (depth==r) {
4         for(int j=0;j<r;j++) out[*count][j]=path[j];
5         (*count)++; return;
6     }
7     for (int i=0; i<n; i++) {
8         if (used[i]) continue;
9         used[i]=1; path[depth]=A[i];
10        perm_visit(A,n,r,depth+1,used,path,out,count);
11        used[i]=0;
12    }
13 }
14 int make_permutations(const int A[], int n, int r, int out[][6]) {
15     int path[6]={0}, count=0;
16     int used[6]={0}; perm_visit(A,n,r,0,used,path,out,&count);
17     return count;
18 }

```

시간 상한 $O(n \cdot n!)$, 결과 수 nPr . 각 결과를 기록할 때 r 개 원소를 복사한다.

개념 및 계산

5명 중 서로 다른 3명에게 금·은·동메달을 하나씩 준다. 한 사람이 여러 메달을 받지 않을 때 경우의 수를 쓰시오.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

5명 중 서로 다른 3명에게 금·은·동메달을 하나씩 준다. 한 사람이 여러 메달을 받지 않을 때 경우의 수를 쓰시오.

정답 60가지

해설

$5P3 = 5 \times 4 \times 3 = 60$ 이다. 누가 금인지 은인지가 달라지면 다른 결과이므로 순열로 센다.

개념 및 계산

서로 다른 음식 5가지 중 서로 다른 2가지를 골라 함께 먹는다. 고르는 순서를 구별하지 않을 때 경우의 수를 쓰시오.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

서로 다른 음식 5가지 중 서로 다른 2가지를 골라 함께 먹는다. 고르는 순서를 구별하지 않을 때 경우의 수를 쓰시오.

정답 10가지

해설

$5C2 = (5 \times 4) \div 2 = 10$ 이다. 짜장면과 짬뽕을 고르는 것과 짬뽕과 짜장면을 고르는 것은 같은 선택이므로 조합으로 센다.

r개 조합 직접 생성

주어진 것: 서로 다른 정수 배열 A(길이 n)와 선택 개수 r. $0 \leq r \leq n \leq 6$ 이다.

반환할 것: r개를 고른 모든 조합. 먼저 고른 원소의 오른쪽에서만 다음 원소를 고르며, 매번 A의 앞쪽부터 시도한다.

입력 순서를 유지하고 같은 조합을 다시 담지 않는다. r=0이면 빈 조합 하나이다. 조합 라이브러리는 쓰지 않는다.

```
make_combinations([3,1,2], 2)
```

```
→ [[3,1],[3,2],[1,2]]
```

작성할 내용

Python: 조합 목록, r=0이면 [[]].

C: out에 한 조합씩 r칸을 쓰고 행 수 반환.

20행이 준비되어 있다. r=0이면 1 반환.

시간 복잡도를 주석으로 쓰시오.

```
Python: make_combinations(A, r)
```

```
C: int make_combinations(const int A[], int n, int r, int out[][6])
```

r개 조합 직접 생성

```
1 def make_combinations(A, r):
2     result, path = [], []
3     def dfs(start):
4         if len(path) == r:
5             result.append(path.copy())
6             return
7         for i in range(start, len(A)):
8             path.append(A[i])
9             dfs(i + 1)
10            path.pop()
11    dfs(0)
12    return result
```

시간 상한 $O(n \cdot 2^n)$, 결과 수 nCr . 미완성 경로의 탐색과 결과 복사도 포함한다.

r개 조합 직접 생성

필요한 표준 헤더: <stdlib.h> <string.h> <limits.h> / 결과 배열은 호출자가 준비

```
1 static void comb_visit(const int A[], int n, int r, int depth, int start, int path[],
2     int out[][6], int *count) {
3     if (depth==r) {
4         for(int j=0;j<r;j++) out[*count][j]=path[j];
5         (*count)++; return;
6     }
7     for (int i=start; i<n; i++) {
8         path[depth]=A[i];
9         comb_visit(A,n,r,depth+1,i+1,path,out,count);
10    }
11 }
12 int make_combinations(const int A[], int n, int r, int out[][6]) {
13     int path[6]={0}, count=0;
14     comb_visit(A,n,r,0,0,path,out,&count);
15     return count;
16 }
```

시간 상한 $O(n \cdot 2^n)$, 결과 수 nCr . 미완성 경로의 탐색과 결과 복사도 포함한다.

개념 및 계산

서로 다른 n 개를 하나도 남기지 않고 모두 나열하는 순열의 개수를 n 에 대한 식으로 쓰시오.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

서로 다른 n 개를 하나도 남기지 않고 모두 나열하는 순열의 개수를 n 에 대한 식으로 쓰시오.

정답 $n!$

해설

첫 자리에 n 가지, 다음에 $n-1$ 가지 식으로 끝까지 곱한 값이다. 외판원 순회처럼 전체 순서를 정하는 문제에서 이 값이 그대로 탐색 공간이 된다.

개념 및 계산

다음은 조합을 만드는 재귀 함수의 일부이다. path는 지금 고른 원소 목록이고 길이는 len(path)이다. 3개 대신 4개를 골라 저장하려면 조건의 어느 숫자를 어떻게 바꾸는가?

```
if len(path) == 3:  
    result.append(path[:])  
    return
```

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

다음은 조합을 만드는 재귀 함수의 일부이다. path는 지금 고른 원소 목록이고 길이는 len(path)이다. 3개 대신 4개를 골라 저장하려면 조건의 어느 숫자를 어떻게 바꾸는가?

```
if len(path) == 3:  
    result.append(path[:])  
    return
```

정답 조건의 3을 4로 바꾼다.

해설

len(path)는 지금까지 선택한 개수이다. 4개를 고른 결과를 저장하려면 len(path)==4일 때 저장하고 돌아오게 한다.

개념 및 계산

어떤 문제를 순열로 셀지 조합으로 셀지 판단할 때 스스로에게 던져야 할 질문을 한 문장으로 쓰시오.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

어떤 문제를 순열로 셀지 조합으로 셀지 판단할 때 스스로에게 던져야 할 질문을 한 문장으로 쓰시오.

정답 뽑은 것들의 자리를 서로 바꾸면 다른 경우가 되는가?

해설

달라지면 순열, 같으면 조합이다. 메달 색이 바뀌면 다른 결과지만 세트 메뉴는 순서를 바꿔도 같은 메뉴다.

문자열의 모든 일치 위치

주어진 것: 영문 문자열 `text`(길이 n , $0 \leq n \leq 50$)와 `pattern`(길이 m , $1 \leq m \leq 10$). 대소문자를 구별한다.
할 일: 시작 위치를 한 칸씩 옮기며 `pattern`과 글자별로 비교한다.
반환할 것: 일치하는 모든 시작 위치를 작은 순서로 담은 목록. 위치는 0부터 세고 겹치는 일치도 포함한다. 없으면 빈 목록이다.

`bf_string_match("AAAAA", "AAA")` → `[0,1,2]`
시작 위치 0, 1, 2에서 겹쳐도 모두 센다.

작성할 내용

Python: 시작 위치 목록을 반환한다.
C: 준비된 `out`(50칸)에 위치를 쓰고 개수를 반환한다. 없으면 0.
시간 복잡도를 주석으로 쓰시오.

Python: `bf_string_match(text, pattern)`

C: `int bf_string_match(const char text[], const char pattern[], int out[])`

문자열의 모든 일치 위치

```
1 def bf_string_match(text, pattern):
2     result = []
3     for i in range(len(text) - len(pattern) + 1):
4         j = 0
5         while j < len(pattern) and text[i+j] == pattern[j]:
6             j += 1
7         if j == len(pattern):
8             result.append(i)
9     return result
```

$n \geq m$: 최악 $O((n-m+1)m)$. $n < m$: Python $O(1)$, C는 `strlen` 때문에 $O(n+m)$.

문자열의 모든 일치 위치

필요한 표준 헤더: <stdlib.h> <string.h> <limits.h> / 결과 배열은 호출자가 준비

```
1 int bf_string_match(const char text[], const char pattern[], int out[]) {
2     int n=(int)strlen(text), m=(int)strlen(pattern), count=0;
3     for(int i=0;i<=n-m;i++) {
4         int j=0;
5         while(j<m && text[i+j]==pattern[j]) j++;
6         if(j==m) out[count++]=i;
7     }
8     return count;
9 }
```

$n \geq m$: 최악 $O((n-m+1)m)$. $n < m$: Python $O(1)$, C는 `strlen` 때문에 $O(n+m)$.

가장 가까운 두 수

주어진 것: 정수 배열 A (길이 n , $2 \leq n \leq 30$). 값은 $-2,000,000,000 \sim 2,000,000,000$ 이며 중복도 가능하다.
 할 일: $i=0,1,\dots,n-2$ 순서로, 각 i 에서 $j=i+1,\dots,n-1$ 순서로 모든 쌍을 검사한다.
 반환할 것: 차이 $|A[i]-A[j]|$ 가 가장 작은 $[A[i],A[j],\text{차이}]$. 차이가 같으면 먼저 찾은 쌍을 유지한다.

입력 `closest_pair([10,3,22,15,17,9])`
 결과 `[10,9,1]`

작성할 내용

Python: [첫 값, 둘째 값, 차이] 반환.
 C: `out[0]`, `out[1]`, `out[2]`에 이 순서로 저장하며 반환값은 없다. 차이는 long long으로 계산한다.
 시간 복잡도를 주석으로 쓰시오.

Python: `closest_pair(A)`
 C: `void closest_pair(const int A[], int n, long long out[3])`

가장 가까운 두 수

```
1 def closest_pair(A):
2     best = None
3     for i in range(len(A)):
4         for j in range(i + 1, len(A)):
5             gap = abs(A[i] - A[j])
6             if best is None or gap < best[2]:
7                 best = [A[i], A[j], gap]
8     return best
```

시간 $O(n^2)$.

가장 가까운 두 수

필요한 표준 헤더: <stdlib.h> <string.h> <limits.h> / 결과 배열은 호출자가 준비

```
1 void closest_pair(const int A[], int n, long long out[3]) {  
2     out[2]=LLONG_MAX;  
3     for(int i=0;i<n;i++) for(int j=i+1;j<n;j++) {  
4         long long diff=llabs((long long)A[i]-A[j]);  
5         if(diff<out[2]) { out[0]=A[i]; out[1]=A[j]; out[2]=diff; }  
6     }  
7 }
```

시간 $O(n^2)$.

비트마스크 부분집합 목록

주어진 것: 서로 다른 정수 배열 A (길이 n , $0 \leq n \leq 8$).

반환할 것: 빈 부분집합을 포함한 모든 부분집합. 비트마스크 0부터 $2^n - 1$ 까지의 순서로 만든다.

오른쪽 0번 비트가 1이면 $A[0]$ 을 고른다. 각 결과 안에서는 A 의 입력 순서를 유지한다. 예: 마스크 5(101)는 $A[0]$, $A[2]$ 를 고른다.

입력 `power_set([1,2,3])`
 결과 `[[[]],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]`

작성할 내용

Python: 부분집합 목록, $A=[]$ 이면 `[[[]]]`.
 C: `out[k]`에 k 번째 부분집합, `sizes[k]`에 그 길이를 쓰고 부분집합 수 반환. 각각 256행/칸이 준비되어 있다. 빈 행은 길이 0. 시간 복잡도를 주석으로 쓰시오.

Python: `power_set(A)`
 C: `int power_set(const int A[], int n, int out[][8], int sizes[])`

비트마스크 부분집합 목록

```
1 def power_set(A):
2     result = []
3     for mask in range(1 << len(A)):
4         subset = []
5         for j in range(len(A)):
6             if mask & (1 << j):
7                 subset.append(A[j])
8         result.append(subset)
9     return result
```

시간 $O(n \cdot 2^n)$.

비트마스크 부분집합 목록

필요한 표준 헤더: <stdlib.h> <string.h> <limits.h> / 결과 배열은 호출자가 준비

```
1 int power_set(const int A[], int n, int out[][8], int sizes[]) {
2     int count=1<<n;
3     for(int mask=0;mask<count;mask++) {
4         sizes[mask]=0;
5         for(int j=0;j<n;j++) if(mask & (1u<<j))
6             out[mask][sizes[mask]++]=A[j];
7     }
8     return count;
9 }
```

시간 $O(n \cdot 2^n)$.

개념 및 계산

텍스트 길이가 n , 패턴 길이가 m 이고 $n \geq m \geq 1$ 이다. 가능한 시작 위치를 모두 검사하며 각 위치의 첫 글자에서 불일치하면 총 문자 비교 횟수는?

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

텍스트 길이가 n , 패턴 길이가 m 이고 $n \geq m \geq 1$ 이다. 가능한 시작 위치를 모두 검사하며 각 위치의 첫 글자에서 불일치하면 총 문자 비교 횟수는?

정답 $n-m+1$ 회

해설

시작 위치는 0부터 $n-m$ 까지이며, 위치마다 한 번 비교한다.

개념 및 계산

차이가 가장 작은 두 수를 best에 저장한다. 처음 $best=(10,3)$ 이고, 이어서 $(10,9)$, $(3,9)$ 를 이 순서로 검사한다. 두 수의 차이의 절댓값이 더 작을 때만 best를 바꾸면 마지막 best는?

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

차이가 가장 작은 두 수를 best에 저장한다. 처음 $\text{best}=(10,3)$ 이고, 이어서 $(10,9)$, $(3,9)$ 를 이 순서로 검사한다. 두 수의 차이의 절댓값이 더 작을 때만 best를 바꾸면 마지막 best는?

정답 $(10, 9)$

해설

처음 차이는 $|10-3|=7$ 이다. $(10,9)$ 의 차이 1이 더 작으므로 바꾼다. $(3,9)$ 의 차이는 6이므로 바꾸지 않는다.

개념 및 계산

원소가 10개인 집합의 부분집합 개수를 쓰시오.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

원소가 10개인 집합의 부분집합 개수를 쓰시오.

정답 1,024개

해설

$2^{10} = 1,024$ 다. 원소가 20개면 약 100만 개, 30개면 약 10억 개가 되어 곧 감당하기 어려워진다.

개념 및 계산

각각 n 번 도는 k 겹 반복(k 는 고정 양의 정수), 서로 다른 n 개 원소의 모든 부분집합, 전체 순열의 후보 수 증가를 각각 빅오로 쓰시오.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

각각 n 번 도는 k 겹 반복(k 는 고정 양의 정수), 서로 다른 n 개 원소의 모든 부분집합, 전체 순열의 후보 수 증가를 각각 빅오로 쓰시오.

정답 $O(n^k)$, $O(2^n)$, $O(n!)$

해설

각각 다항식, 지수, 팩토리얼 증가이다. 후보 한 개를 검사하는 비용은 별도로 곱한다.

개념 및 계산

$n=3$ 일 때 $n!$, 2^n , n^n 의 값을 각각 쓰시오.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

$n=3$ 일 때 $n!$, 2^n , n^n 의 값을 각각 쓰시오.

정답 **6, 8, 27**

해설

$3! = 3 \times 2 \times 1 = 6$, $2^3 = 8$, $3^3 = 27$ 이다.

개념 및 계산

각 자리에 0~9를 중복해서 쓸 수 있고 앞자리 0도 허용하는 8자리 비밀번호의 경우의 수를 쓰시오.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

각 자리에 0~9를 중복해서 쓸 수 있고 앞자리 0도 허용하는 8자리 비밀번호의 경우의 수를 쓰시오.

정답 1억 가지 (10^8)

해설

자리마다 10가지가 곱해진다. 초당 1만 번씩 시도한다면 $1\text{억} \div 1\text{만} = 1\text{만 초}$, 약 3시간이 걸린다.

개념 및 계산

원소가 30개인 집합의 부분집합 개수를 거듭제곱식으로 쓰시오. 십진수로 계산하지 않아도 된다.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

원소가 30개인 집합의 부분집합 개수를 거듭제곱식으로 쓰시오. 십진수로 계산하지 않아도 된다.

정답 2^{30} 개

해설

각 원소를 포함하거나 제외하는 두 선택이 있으므로 2를 30번 곱한 2^{30} 개이다.

출발점을 고정한 외판원 순회

주어진 것: 도시 n 개($2 \leq n \leq 8$)의 비용 행렬 dist . 도시 번호는 $0 \sim n-1$ 이고 $\text{dist}[i][j]$ 는 i 에서 j 로 가는 비용이다.

반환할 것: 0에서 출발해 다른 도시를 한 번씩 방문한 뒤 0으로 돌아오는 최소 비용.

모든 이동이 가능하다. 같은 도시의 비용은 0, 나머지는 $1 \sim 1,000$ 이며 반대 방향의 비용은 다를 수 있다.

`tsp_min([[0,2,9],[1,0,3],[4,8,0]])` → 9

$0 \rightarrow 1 \rightarrow 2 \rightarrow 0$ 의 비용은 $2+3+4=9$ 이다.

작성할 내용

Python: $n \times n$ 목록을 받는다.

C: 한 행이 8칸인 배열의 앞 n 행· n 열을 사용한다. 두 언어 모두 최소 비용을 정수로 반환한다.

시간 복잡도를 주석으로 쓰시오.

Python: `tsp_min(dist)`

C: `int tsp_min(const int dist[][8], int n)`

출발점을 고정한 외판원 순회

```
1 def tsp_min(dist):
2     n = len(dist)
3     used = [False] * n
4     used[0] = True
5     def dfs(last, depth, cost):
6         if depth == n:
7             return cost + dist[last][0]
8         best = float('inf')
9         for city in range(1, n):
10            if not used[city]:
11                used[city] = True
12                value = dfs(city, depth + 1, cost + dist[last][city])
13                best = min(best, value)
14                used[city] = False
15        return best
16    return dfs(0, 1, 0)
```

후보 순서 $(n-1)!$ 개, 이 구현의 상한 $O(n \cdot (n-1)!)$.

출발점을 고정한 외판원 순회

필요한 표준 헤더: <stdlib.h> <string.h> <limits.h> / 결과 배열은 호출자가 준비

```

1 static void tsp_visit(const int d[][8], int n, int depth, int last,
2                       int used[], int cost, int *best) {
3     if(depth==n) {
4         int total=cost+d[last][0];
5         if(total<*best) *best=total;
6         return;
7     }
8     for(int next=1;next<n;next++) if(!used[next]) {
9         used[next]=1;
10        tsp_visit(d,n,depth+1,next,used,cost+d[last][next],best);
11        used[next]=0;
12    }
13 }
14 int tsp_min(const int dist[][8], int n) {
15     int used[8]={1}, best=INT_MAX;
16     tsp_visit(dist,n,1,0,used,0,&best);
17     return best;
18 }

```

후보 순서 $(n-1)!$ 개, 이 구현의 상한 $O(n \cdot (n-1)!)$.

개념 및 계산

서로 다른 n 개 원소의 모든 부분집합을 만든다. 기존에 없던 원소 하나를 추가하면 부분집합 수는 몇 배가 되는가?

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

서로 다른 n 개 원소의 모든 부분집합을 만든다. 기존에 없던 원소 하나를 추가하면 부분집합 수는 몇 배가 되는가?

정답 2배

해설

새 원소를 포함하는 경우와 포함하지 않는 경우로 갈라지므로 정확히 두 배가 된다. 이 두 배가 반복되어 2^n 이 된다.

개념 및 계산

서로 다른 원소 n 개의 모든 부분집합 2^n 개를 검사한다. 최대 1,048,576개까지 검사할 수 있을 때 원소 수 n 의 최댓값은? 참고: $2^{20}=1,048,576$ 이며 n 이 1 늘면 후보 수는 2배다.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

서로 다른 원소 n 개의 모든 부분집합 2^n 개를 검사한다. 최대 1,048,576개까지 검사할 수 있을 때 원소 수 n 의 최댓값은? 참고: $2^{20}=1,048,576$ 이며 n 이 1 늘면 후보 수는 2배다.

정답 20

해설

$n=20$ 이면 한도와 같고, $n=21$ 이면 한도의 두 배가 되므로 최대는 20이다.

개념 및 계산

서로 다른 원소 n 개를 모두 나열하는 순열 $n!$ 개를 검사한다. 최대 1,000만 개까지 검사할 수 있을 때 원소 수 n 의 최댓값은? 참고: $10!=3,628,800$, $11!=39,916,800$ 이다.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

서로 다른 원소 n 개를 모두 나열하는 순열 $n!$ 개를 검사한다. 최대 1,000만 개까지 검사할 수 있을 때 원소 수 n 의 최댓값은? 참고: $10!=3,628,800$, $11!=39,916,800$ 이다.

정답 10

해설

$10!$ 은 한도 이하이고 $11!$ 은 한도를 넘는다. 팩토리얼 값은 문제에 제공된 수를 비교하면 된다.

개념 및 계산

제한 시간에 후보를 최대 2^{20} 개까지 검사할 수 있어, 모든 부분집합을 검사할 수 있는 최대 원소 수는 20개이다. 후보당 비용이 같고 컴퓨터가 2배 빨라지면, 같은 시간의 최대 원소 수는 몇 개 늘어나는가?

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

제한 시간에 후보를 최대 2^{20} 개까지 검사할 수 있어, 모든 부분집합을 검사할 수 있는 최대 원소 수는 20개이다. 후보당 비용이 같고 컴퓨터가 2배 빨라지면, 같은 시간의 최대 원소 수는 몇 개 늘어나는가?

정답 1개

해설

검사 가능한 후보 수가 $2 \times 2^{20} = 2^{21}$ 개가 된다. 원소 수는 20개에서 21개로 한 개 늘어난다.

개념 및 계산

원소를 하나씩 고르며 탐색하다가, 현재 선택을 더 이어도 조건을 만족할 수 없으면 그 갈래를 중단하고 이전 선택으로 돌아간다. 이 기법의 이름은?

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

원소를 하나씩 고르며 탐색하다가, 현재 선택을 더 이어도 조건을 만족할 수 없으면 그 갈래를 중단하고 이전 선택으로 돌아간다. 이 기법의 이름은?

정답 백트래킹

해설

백트래킹이다. 불가능한 갈래를 더 만들지 않는 가지치기를 사용하고, 다른 선택을 시도하기 위해 이전 단계로 돌아간다.

개념 및 계산

병합 정렬처럼 문제를 작은 문제로 나누어 각각 해결하고 결과를 합치는 기법의 이름을 쓰시오.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

병합 정렬처럼 문제를 작은 문제로 나누어 각각 해결하고 결과를 합치는 기법의 이름을 쓰시오.

정답 분할 정복

해설

문제를 분할하고 작은 문제를 해결한 뒤 결합한다.

개념 및 계산

n 은 0 이상의 정수이며 $F(0)=0$, $F(1)=1$, $n \geq 2$ 에서 $F(n)=F(n-1)+F(n-2)$ 이다. ① 같은 값을 매번 재귀로 다시 계산 ② 처음 구한 값을 저장하고 재사용할 때, $F(n)$ 계산 시간의 상한을 각각 쓰시오. 덧셈·기록 조회는 $O(1)$ 이고 기록은 처음에 비어 있다.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

n 은 0 이상의 정수이며 $F(0)=0$, $F(1)=1$, $n \geq 2$ 에서 $F(n)=F(n-1)+F(n-2)$ 이다. ① 같은 값을 매번 재귀로 다시 계산 ② 처음 구한 값을 저장하고 재사용할 때, $F(n)$ 계산 시간의 상한을 각각 쓰시오. 덧셈·기록 조회는 $O(1)$ 이고 기록은 처음에 비어 있다.

정답 ① 단순 재귀 $O(2^n)$ ② 메모이제이션 $O(n)$

해설

덧셈을 상수 시간으로 가정한다. 메모이제이션은 각 $F(k)$ 를 한 번만 계산한다.

개념 및 계산

$F(5)$ 를 구하는 중 이미 구한 $F(3)$ 가 다시 필요해졌다. 메모이제이션을 쓸 때 $F(3)$ 의 중복 계산을 어떻게 피하는지 한 문장으로 쓰시오.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

$F(5)$ 를 구하는 중 이미 구한 $F(3)$ 가 다시 필요해졌다. 메모이제이션을 쓸 때 $F(3)$ 의 중복 계산을 어떻게 피하는지 한 문장으로 쓰시오.

정답 처음 구한 $F(3)$ 를 저장해 두고, 다시 필요하면 저장한 값을 읽는다.

해설

같은 인수의 계산 결과를 기억해 두면 다시 재귀 호출을 펼칠 필요가 없다. 저장한 답을 그대로 재사용한다.

개념 및 계산

각각 정확히 n 번 도는 삼중 반복문의 가장 안쪽 연산은 $n=100$ 일 때 몇 번 실행되는가?

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

각각 정확히 n 번 도는 삼중 반복문의 가장 안쪽 연산은 $n=100$ 일 때 몇 번 실행되는가?

정답 1,000,000번

해설

$100 \times 100 \times 100 = 1,000,000$ 이다.

개념 및 계산

다음 작업에 맞는 도구를 고르시오. $n \geq 1$, $1 \leq r \leq n$ 이며 원소는 서로 다르다.

보기: 단일 for / 이중 for / combinations / permutations / 비트마스크

- ① 1~n의 각 수가 n의 약수인지 검사 ② 모든 $i < j$ 쌍 검사
- ③ r개를 순서 없이 선택 ④ r개를 순서 있게 나열
- ⑤ 각 원소를 넣거나 빼는 모든 부분집합 생성

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

다음 작업에 맞는 도구를 고르시오. $n \geq 1$, $1 \leq r \leq n$ 이며 원소는 서로 다르다.

보기: 단일 for / 이중 for / combinations / permutations / 비트마스크

- ① 1~n의 각 수가 n의 약수인지 검사 ② 모든 $i < j$ 쌍 검사
- ③ r개를 순서 없이 선택 ④ r개를 순서 있게 나열
- ⑤ 각 원소를 넣거나 빼는 모든 부분집합 생성

정답 ① 단일 for ② 이중 for ③ combinations
④ permutations ⑤ 비트마스크

해설

①은 수 하나, ②는 위치 두 개를 순회한다. ③과 ④는 순서 구별 여부가 다르다. ⑤는 각 비트로 원소를 넣을지 뺄지를 나타낸다.

0 1 배낭 구현

주어진 것: 길이가 같은 weights, values와 용량 capacity. 길이 n은 0~16, 무게·가치는 1~100, capacity는 0~300이다.

i번 물건의 무게는 weights[i], 가치는 values[i]이다. 각 물건은 최대 한 번만 고른다.

반환할 것: 총무게가 capacity 이하인 선택 중 최대 총가치. 아무것도 고르지 않으면 0이다.

비트마스크로 모든 선택을 검사한다.

knapsack([4,6,3,5], [5,8,3,6], 10) → 13

0번과 1번 물건: 총무게 10, 총가치 13.

작성할 내용

Python과 C 모두 최대 총가치를 반환한다.

C의 n은 두 배열의 공통 길이이다. 선택한 물건 목록은 반환하지 않는다.

시간 복잡도를 주석으로 쓰시오.

Python: knapsack(weights, values, capacity)

C: int knapsack(const int weights[], const int values[], int n, int capacity)

0 1 배낭 구현

```
1 def knapsack(weights, values, capacity):
2     best = 0
3     for mask in range(1 << len(weights)):
4         weight, value = 0, 0
5         for j in range(len(weights)):
6             if mask & (1 << j):
7                 weight += weights[j]
8                 value += values[j]
9             if weight <= capacity and value > best:
10                 best = value
11     return best
```

시간 $O(n \cdot 2^n)$.

0 1 배낭 구현

필요한 표준 헤더: <stdlib.h> <string.h> <limits.h> / 결과 배열은 호출자가 준비

```
1 int knapsack(const int weights[], const int values[], int n, int capacity) {
2     int best=0;
3     for(unsigned mask=0;mask<(1u<<n);mask++) {
4         int weight=0, value=0;
5         for(int j=0;j<n;j++) if(mask & (1u<<j)) {weight+=weights[j]; value+=values[j];}
6         if(weight<=capacity && value>best) best=value;
7     }
8     return best;
9 }
```

시간 $O(n \cdot 2^n)$.

개념 및 계산

회문(palindrome)이 무엇인지 한 문장으로 쓰시오.

답을 값·식 또는 한두 문장으로 쓰시오.

개념 및 계산

회문(palindrome)이 무엇인지 한 문장으로 쓰시오.

정답 앞에서 읽으나 뒤에서 읽으나 같은 문자열

해설

level과 noon이 예이다. 양 끝의 문자를 차례로 비교하여 확인할 수 있다.